

fetch_dist_osrm

April 12, 2026

1 Define Bangalore Zones

We define 16 major Bangalore zones with: - **Zone ID** - **Zone Name** - **Latitude & Longitude**

These coordinates will be used to compute real road distances.

```
[1]: # Fetch REAL road distances between Bangalore zones using the FREE ↪
      ↪OpenStreetMap OSRM API.

import requests
import pandas as pd
import numpy as np

zones = {
    0: {"name": "Whitefield",      "lat": 12.9698, "lon": 77.7500},
    1: {"name": "Koramangala",     "lat": 12.9352, "lon": 77.6245},
    2: {"name": "Indiranagar",     "lat": 12.9784, "lon": 77.6408},
    3: {"name": "Hebbal",         "lat": 13.0352, "lon": 77.5970},
    4: {"name": "Marathahalli",    "lat": 12.9591, "lon": 77.6972},
    5: {"name": "Electronic City", "lat": 12.8399, "lon": 77.6770},
    6: {"name": "Jayanagar",       "lat": 12.9308, "lon": 77.5838},
    7: {"name": "Rajajinagar",     "lat": 12.9856, "lon": 77.5533},
    8: {"name": "Yeshwanthpur",    "lat": 13.0237, "lon": 77.5406},
    9: {"name": "BTM Layout",      "lat": 12.9166, "lon": 77.6101},
    10: {"name": "HSR Layout",     "lat": 12.9116, "lon": 77.6389},
    11: {"name": "Bannerghatta Rd", "lat": 12.8933, "lon": 77.5969},
    12: {"name": "Yelahanka",      "lat": 13.1007, "lon": 77.5963},
    13: {"name": "Sarjapur Road",  "lat": 12.9010, "lon": 77.6810},
    14: {"name": "MG Road",        "lat": 12.9756, "lon": 77.6086},
    15: {"name": "Banashankari",   "lat": 12.9255, "lon": 77.5468},
}
```

2 Extract Zone Identifiers

We create: - **ids** → list of zone indices - **names** → list of zone names

These help in labeling the distance matrix.

```
[2]: ids    = list(zones.keys())
names = [zones[i]["name"] for i in ids]
```

3 Build Coordinate String

We construct a coordinate string in the format:

lon,lat;lon,lat;...

This format is required by the **OSRM API** for multi-point routing queries.

```
[3]: # Build coordinate string: lon,lat;lon,lat;...
coords_str = ";".join([f"{zones[i]['lon']},{zones[i]['lat']}" for i in ids])
coords_str
```

```
[3]: '77.75,12.9698;77.6245,12.9352;77.6408,12.9784;77.597,13.0352;77.6972,12.9591;77.
.677,12.8399;77.5838,12.9308;77.5533,12.9856;77.5406,13.0237;77.6101,12.9166;77.
6389,12.9116;77.5969,12.8933;77.5963,13.1007;77.681,12.901;77.6086,12.9756;77.54
68,12.9255'
```

4 Fetch and Process Distance Matrix

We use the **OpenStreetMap OSRM API** to compute real-world road distances.

Process: 1. Create API request using `/table/v1/driving/` 2. Send GET request to OSRM server
3. Retrieve response in JSON format 4. Extract distance matrix (in meters) 5. Convert **meters**
→ **kilometers** 6. Round values to 2 decimal places

Output: - A NumPy distance matrix (in km) ready for analysis

```
[4]: url = (
    f"http://router.project-osrm.org/table/v1/driving/{coords_str}"
    f"?annotations=distance"
)

response = requests.get(url, timeout=60)

if response.status_code == 200:
    data = response.json()

    # Distance matrix: meters → kilometers
    dist_km = np.round(np.array(data["distances"]) / 1000, 2)

    # Save distance matrix
    df_dist = pd.DataFrame(dist_km, index=names, columns=names)
    df_dist.index.name = "Zone"
    df_dist.to_csv("distance_matrix_osrm.csv")

    print("\nSample distances (km): top-left 5x5:")
```

```

print(df_dist.iloc[:5, :5].to_string())

else:
    print(f"Error {response.status_code}: {response.text}")
    print("Try again.")

```

Sample distances (km): top-left 5x5:

	Whitefield	Koramangala	Indiranagar	Hebbal	Marathahalli
Zone					
Whitefield	0.00	16.94	14.61	25.97	7.30
Koramangala	17.25	0.00	5.99	15.35	10.77
Indiranagar	14.58	6.00	0.00	11.65	8.10
Hebbal	23.13	15.69	11.32	0.00	19.38
Marathahalli	7.02	11.19	8.86	20.22	0.00

[]:

preprocess

April 12, 2026

```
[1]: import json
import pandas as pd
import numpy as np
import os
```

1 Load and Inspect Bangalore Ward GeoJSON

This cell loads the `bangalore_wards.json` file and inspects its structure.

It checks: - whether the file is a dictionary or list, - the top-level keys, - whether the file is in **GeoJSON** format, - the total number of features, - and sample ward properties from the first few entries.

This helps confirm that the ward boundary/mapping file has been loaded correctly before further preprocessing.

```
[2]: with open('bangalore_wards.json') as f:
    data = json.load(f)

print(f"Type: {type(data)}")
if isinstance(data, dict):
    print(f"Keys: {list(data.keys())}")
elif isinstance(data, list):
    print(f"Length: {len(data)}")
    print(f"First item keys: {list(data[0].keys()) if isinstance(data[0], dict)
    ↪ else data[0]}")

# Check if it's GeoJSON
if isinstance(data, dict) and 'type' in data:
    print(f"\nGeoJSON type: {data['type']}")
    if 'features' in data:
        print(f"Number of features: {len(data['features'])}")
        f0 = data['features'][0]
        print(f"Feature keys: {list(f0.keys())}")
        print(f"Properties: {f0['properties']}")
        print(f"\nFirst 5 features:")
        for feat in data['features'][:5]:
            print(f"  {feat['properties']}")
```

```
Type: <class 'dict'>
Keys: ['type', 'features']
```

```
GeoJSON type: FeatureCollection
```

```
Number of features: 198
```

```
Feature keys: ['type', 'properties', 'geometry']
```

```
Properties: {'WARD_NO': '2', 'WARD_NAME': 'Chowdeswari Ward', 'MOVEMENT_ID': '1', 'DISPLAY_NAME': 'Unnamed Road, Bengaluru'}
```

```
First 5 features:
```

```
{'WARD_NO': '2', 'WARD_NAME': 'Chowdeswari Ward', 'MOVEMENT_ID': '1',
'DISPLAY_NAME': 'Unnamed Road, Bengaluru'}
{'WARD_NO': '3', 'WARD_NAME': 'Atturu', 'MOVEMENT_ID': '2', 'DISPLAY_NAME':
'9th Cross Bhel Layout, Adityanagar, Vidyaranyapura, Bengaluru'}
{'WARD_NO': '4', 'WARD_NAME': 'Yelahanka Satellite Town', 'MOVEMENT_ID': '3',
'DISPLAY_NAME': '15th A Cross Road, Yelahanka Satellite Town, Yelahanka,
Bengaluru'}
{'WARD_NO': '51', 'WARD_NAME': 'Vijnanapura', 'MOVEMENT_ID': '4',
'DISPLAY_NAME': 'SP Naidu Layout 4th Cross Street, SP Naidu Layout, Dooravani
Nagar, Bengaluru'}
{'WARD_NO': '53', 'WARD_NAME': 'Basavanapura', 'MOVEMENT_ID': '5',
'DISPLAY_NAME': 'Medahalli Kadugodi Road, Bharathi Nagar, Krishnarajapura,
Bengaluru'}
```

2 Build Ward Mapping Table

This cell extracts relevant metadata from the GeoJSON features and builds a structured mapping table.

The generated dataframe `df_map` contains: - `movement_id` - `ward_no` - `ward_name` - `display_name`

The table is sorted by `movement_id`.

```
[3]: # Build full mapping table
rows = []
for feat in data['features']:
    p = feat['properties']
    rows.append({
        'movement_id': int(p['MOVEMENT_ID']),
        'ward_no':      p['WARD_NO'],
        'ward_name':    p['WARD_NAME'],
        'display_name': p['DISPLAY_NAME']
    })

df_map = pd.DataFrame(rows).sort_values('movement_id')
df_map
```

```
[3]:      movement_id ward_no      ward_name \
0          1          2      Chowdeswari Ward
1          2          3          Atturu
2          3          4  Yelahanka Satellite Town
3          4          51      Vijnanapura
4          5          53      Basavanapura
..      ...      ...
193      194      172          Madivala
194      195          26      Ramamurthy Nagar
195      196          25          Horamavu
196      197          86      Marathahalli
197      198          198      Hemmigeepura

      display_name
0          Unnamed Road, Bengaluru
1      9th Cross Bhel Layout, Adityanagar, Vidyaranya...
2      15th A Cross Road, Yelahanka Satellite Town, Y...
3      SP Naidu Layout 4th Cross Street, SP Naidu Lay...
4      Medahalli Kadugodi Road, Bharathi Nagar, Krish...
..
193  0 1st B Cross Road, Cashier Layout, 1st Stage,...
194  Kalkere-Agara Main Road, Horamavu Agara, Kalke...
195  0 Horamavu Agara Main Road, 1st Block, Mallapp...
196  0 3rd Cross Road, Manjunatha Layout, Marathaha...
197          BGS Road, Kodipalya, Bengaluru

[198 rows x 4 columns]
```

3 Map Selected Bangalore Zones to Representative Ward IDs

This cell defines the mapping between the selected Bangalore study zones and their corresponding ward-based `movement_id` values.

The workflow includes: - assigning one representative `movement_id` to each zone, - loading all four quarterly hourly aggregate traffic files, - combining them into a single dataframe, - filtering the data to retain only trips between the selected 16 zones, - removing self-loop trips where source and destination are the same, - adding readable source and destination zone names, - merging inter-zone distance values from the OSRM distance matrix, - and saving the filtered dataset as `uber_bangalore_2018_processed.csv`.

```
[4]: # Our 15 zones + depot → best matching ward IDs
zone_mapping = {
    'Whitefield':      [167, 168, 169, 170],    # Garudachar Playa, Kadugodi,
    ↪ Hagadur, Dodd Nekkundi
    'Koramangala':      [157],                  # Koramangala
    'Indiranagar':      [80],                   # Hoysala Nagar (Indiranagar)
```

```

    'Hebbal': [55, 56], # Hebbala, Vishwanath
    ↪Nagenahalli
    'Marathahalli': [197], # Marathahalli
    'Electronic City': [172, 175], # Bommanahalli area
    'Jayanagar': [140, 149, 150], # Jayanagar, Karisandra
    ↪Yediyur
    'Rajajinagar': [109], # Rajaji Nagar
    'Yeshwanthpur': [25], # Yeshwanthpura
    'BTM Layout': [161, 190], # BTM Layout, Gurappanapalya
    'HSR Layout': [171], # HSR Layout
    'Bannerghatta Rd': [179, 183], # Arakere, Gottigere
    'Yelahanka': [3, 186], # Yelahanka Satellite Town
    ↪Kempegowda Ward
    'Sarjapur Road': [165, 166], # Varthuru, Bellanduru
    'MG Road': [89, 94], # Shantala Nagar, Gandhinagar
    'Banashankari': [144, 145, 146, 151], # Girinagar, Katriguppe
    ↪Vidyapeeta, Banashankari Temple
}

# Use the representative (first) movement_id for each zone
primary_ids = {zone: ids[0] for zone, ids in zone_mapping.items()}
print("--- ZONE → MOVEMENT ID MAPPING ---")
for zone, mid in primary_ids.items():
    ward = df_map[df_map['movement_id'] == mid]['ward_name'].values[0]
    print(f" {zone:<22} → movement_id: {mid:>3} ({ward})")

# Load all 4 quarters and filter
print("\n--- LOADING ALL 4 QUARTERS ---")
dfs = []
for q in [1, 2, 3, 4]:
    path = f'bangalore-wards-2018-{q}-All-HourlyAggregate.csv'
    df_q = pd.read_csv(path)
    df_q['quarter'] = q
    dfs.append(df_q)
    print(f" Q{q}: {len(df_q):>7,} rows loaded")

df_all = pd.concat(dfs, ignore_index=True)
print(f"\n TOTAL: {len(df_all):,} rows combined")

# Filter to our 16 zone IDs
selected_ids = list(primary_ids.values())
df_filtered = df_all[
    df_all['sourceid'].isin(selected_ids) &
    df_all['dstid'].isin(selected_ids) &
    (df_all['sourceid'] != df_all['dstid'])
].copy()

```

```

print(f"\n--- AFTER FILTERING TO 16 ZONES ---")
print(f"  Rows: {len(df_filtered):,}")
print(f"  Unique OD pairs: {df_filtered.groupby(['sourceid','dstid']).ngroups}")
print(f"  Expected OD pairs: {16*15} (16 zones × 15 directions)")

# Add zone names
id_to_zone = {v: k for k, v in primary_ids.items()}
df_filtered['source_name'] = df_filtered['sourceid'].map(id_to_zone)
df_filtered['dest_name']   = df_filtered['dstid'].map(id_to_zone)

# Add distance from real matrix
dist_df = pd.read_csv('distance_matrix_osrm.csv', index_col=0)
def get_dist(row):
    try: return dist_df.loc[row['source_name'], row['dest_name']]
    except: return np.nan
df_filtered['distance_km'] = df_filtered.apply(get_dist, axis=1)

# Save
df_filtered.to_csv('uber_bangalore_2018_processed.csv', index=False)
print(f"\n Saved: uber_bangalore_2018_processed.csv  ({len(df_filtered):,}
↪rows)")

```

--- ZONE → MOVEMENT ID MAPPING ---

Whitefield	→ movement_id: 167	(Garudachar Playa)
Koramangala	→ movement_id: 157	(Koramangala)
Indiranagar	→ movement_id: 80	(Hoysala Nagar)
Hebbal	→ movement_id: 55	(Hebbala)
Marathahalli	→ movement_id: 197	(Marathahalli)
Electronic City	→ movement_id: 172	(Bommanahalli)
Jayanagar	→ movement_id: 140	(Jayanagar)
Rajajinagar	→ movement_id: 109	(Rajaji Nagar)
Yeshwanthpur	→ movement_id: 25	(Yeshwanthpura)
BTM Layout	→ movement_id: 161	(BTM Layout)
HSR Layout	→ movement_id: 171	(HSR Layout)
Bannerghatta Rd	→ movement_id: 179	(Arakere)
Yelahanka	→ movement_id: 3	(Yelahanka Satellite Town)
Sarjapur Road	→ movement_id: 165	(Varthuru)
MG Road	→ movement_id: 89	(Shantala Nagar)
Banashankari	→ movement_id: 144	(Girinagar)

--- LOADING ALL 4 QUARTERS ---

Q1: 824,960 rows loaded
 Q2: 842,326 rows loaded
 Q3: 831,191 rows loaded
 Q4: 825,319 rows loaded

TOTAL: 3,323,796 rows combined

--- AFTER FILTERING TO 16 ZONES ---

Rows: 22,623

Unique OD pairs: 240

Expected OD pairs: 240 (16 zones × 15 directions)

Saved: uber_bangalore_2018_processed.csv (22,623 rows)

```
[5]: df_filtered.reset_index(inplace=True,drop=True)
df_filtered
```

```
[5]:
```

	sourceid	dstid	hod	mean_travel_time	standard_deviation_travel_time	\
0	55	157	0	1363.10	255.38	
1	25	55	9	1301.06	416.42	
2	80	109	20	2973.61	731.30	
3	144	157	18	3207.95	898.67	
4	140	197	18	3530.07	1012.10	
...	...	...	...	...	...	
22618	197	179	3	1593.91	249.73	
22619	197	179	8	2777.05	1086.17	
22620	197	179	13	2570.38	539.38	
22621	197	179	18	4128.97	1261.27	
22622	197	179	23	2010.60	442.58	

	geometric_mean_travel_time	geometric_standard_deviation_travel_time	\
0	1343.56	1.18	
1	1243.27	1.34	
2	2896.02	1.25	
3	3096.49	1.30	
4	3391.62	1.33	
...	...	...	
22618	1575.67	1.16	
22619	2595.35	1.43	
22620	2524.84	1.20	
22621	3939.59	1.36	
22622	1967.71	1.22	

	quarter	source_name	dest_name	distance_km
0	1	Hebbal	Koramangala	15.69
1	1	Yeshwanthpur	Hebbal	9.79
2	1	Indiranagar	Rajajinagar	11.60
3	1	Banashankari	Koramangala	11.99
4	1	Jayanagar	Marathahalli	17.23
...	...	...	...	...
22618	4	Marathahalli	Bannerghatta Rd	19.25
22619	4	Marathahalli	Bannerghatta Rd	19.25
22620	4	Marathahalli	Bannerghatta Rd	19.25
22621	4	Marathahalli	Bannerghatta Rd	19.25

22622

4 Marathahalli Bannerghatta Rd

19.25

[22623 rows x 11 columns]

4 Define Zone List and Zone Categories

This cell defines: - the ordered list of selected Bangalore zones, and - the functional category of each zone.

The categories include: - **IT** - **Commercial** - **Residential** - **Industrial**

```
[6]: ZONES = [
    "Whitefield", "Koramangala", "Indiranagar", "Hebbal",
    "Marathahalli", "Electronic City", "Jayanagar", "Rajajinagar",
    "Yeshwanthpur", "BTM Layout", "HSR Layout", "Bannerghatta Rd",
    "Yelahanka", "Sarjapur Road", "MG Road", "Banashankari"
]
ZONE_TYPE = {
    "Whitefield":"IT",          "Koramangala":"Commercial", "Indiranagar":
    ↪"Commercial",
    "Hebbal":"Residential", "Marathahalli":"IT",          "Electronic City":"IT",
    "Jayanagar":"Residential", "Rajajinagar":"Residential",
    "Yeshwanthpur":"Industrial",
    "BTM Layout":"Residential", "HSR Layout":"Commercial", "Bannerghatta Rd":
    ↪"Residential",
    "Yelahanka":"Residential", "Sarjapur Road":"IT",      "MG Road":
    ↪"Commercial",
    "Banashankari":"Residential",
}
```

5 Final Preprocessing and Feature Engineering

The preprocessing steps include: - loading the filtered travel data, - removing self-loop records, - converting mean and standard deviation travel times from seconds to minutes, - filtering out unreliable observations using the coefficient of variation (CV), - removing very short-duration trips, - assigning source and destination zone categories, - generating one-hot encoded zone-type features, - creating temporal features such as morning peak, evening peak, off-peak, cyclic hour encoding, and monsoon indicators, - computing a speed-based proxy using distance and travel time, - encoding source and destination zones as numeric IDs, - selecting the final set of relevant features, - and saving the cleaned dataset as `data_processed.csv`.

The resulting dataframe is the final structured dataset used for analysis and predictive modeling.

```
[7]: def preprocess():
    df = pd.read_csv("uber_bangalore_2018_processed.csv")
    print(f" Rows loaded: {len(df):,}")
```

```

# Remove self-loops
df = df[df["source_name"] != df["dest_name"]].copy()

# Convert to minutes
df['mean_travel_time_min'] = (df['mean_travel_time'] / 60).round(2)
df['std_travel_time_min'] = (df['standard_deviation_travel_time'] / 60).
↳round(2)

# Remove unreliable records (CV > 1.0)
df["cv"] = df["std_travel_time_min"] / df["mean_travel_time_min"]
before = len(df)
df = df[df["cv"] <= 1.0].copy()
print(f" Removed {before - len(df)} high-CV records")

# Remove very short records
df = df[df["mean_travel_time_min"] >= 1.0].copy()

# Zone type features
df["zone_type_src"] = df["source_name"].map(ZONE_TYPE)
df["zone_type_dst"] = df["dest_name"].map(ZONE_TYPE)
for zt in ["IT", "Commercial", "Residential", "Industrial"]:
    df[f"src_{zt.lower()}"] = (df["zone_type_src"] == zt).astype(int)
    df[f"dst_{zt.lower()}"] = (df["zone_type_dst"] == zt).astype(int)

# Feature Engineering (Temporal features)
df["is_peak_morning"] = df["hod"].isin([7,8,9,10]).astype(int)
df["is_peak_evening"] = df["hod"].isin([17,18,19,20]).astype(int)
df["is_off_peak"] = (~df["hod"].isin([7,8,9,10,17,18,19,20])).
↳astype(int)
df["hod_sin"] = np.sin(2 * np.pi * df["hod"] / 24).round(6)
df["hod_cos"] = np.cos(2 * np.pi * df["hod"] / 24).round(6)
df["is_monsoon"] = df["quarter"].isin([2,3]).astype(int)
df["speed_proxy"] = (df["distance_km"] / (df["mean_travel_time_min"] / 60)).
↳round(4)

# Numeric zone IDs
zone_to_id = {z: i for i, z in enumerate(ZONES)}
df["source_id"] = df["source_name"].map(zone_to_id)
df["dest_id"] = df["dest_name"].map(zone_to_id)

# Select and save final columns
feature_cols = [
    "source_id", "source_name", "dest_id", "dest_name", "distance_km",
↳"mean_travel_time_min", "std_travel_time_min",
    "hod", "is_peak_morning", "is_peak_evening", "is_off_peak",
↳"is_monsoon", "quarter", "speed_proxy", "src_it",

```

```

        "src_commercial", "src_residential", "src_industrial", "dst_it",
        ↪ "dst_commercial", "dst_residential",
        "dst_industrial", "hod_sin", "hod_cos"
    ]
    df_final = df[feature_cols].dropna().copy()
    df_final.to_csv("data_processed.csv", index=False)
    return df_final

```

```
preprocess()
```

Rows loaded: 22,623

Removed 44 high-CV records

```

[7]:
   source_id  source_name  dest_id  dest_name  distance_km \
0          3      Hebbal      1    Koramangala      15.69
1          8  Yeshwanthpur      3      Hebbal       9.79
2          2   Indiranagar      7   Rajajinagar      11.60
3         15  Banashankari      1    Koramangala      11.99
4          6    Jayanagar      4   Marathahalli      17.23
...      ...      ...      ...      ...      ...
22618       4   Marathahalli      11  Bannerghatta Rd      19.25
22619       4   Marathahalli      11  Bannerghatta Rd      19.25
22620       4   Marathahalli      11  Bannerghatta Rd      19.25
22621       4   Marathahalli      11  Bannerghatta Rd      19.25
22622       4   Marathahalli      11  Bannerghatta Rd      19.25

   mean_travel_time_min  std_travel_time_min  hod  is_peak_morning \
0                22.72                4.26    0                0
1                21.68                6.94    9                1
2                49.56               12.19   20                0
3                53.47               14.98   18                0
4                58.83               16.87   18                0
...      ...      ...      ...      ...
22618              26.57                4.16    3                0
22619              46.28               18.10    8                1
22620              42.84                8.99   13                0
22621              68.82               21.02   18                0
22622              33.51                7.38   23                0

   is_peak_evening  ...  src_it  src_commercial  src_residential \
0                0  ...      0                0                1
1                0  ...      0                0                0
2                1  ...      0                1                0
3                1  ...      0                0                1
4                1  ...      0                0                1
...      ...      ...      ...      ...      ...
22618              0  ...      1                0                0

```

22619	0	...	1	0	0
22620	0	...	1	0	0
22621	1	...	1	0	0
22622	0	...	1	0	0

	src_industrial	dst_it	dst_commercial	dst_residential	\
0	0	0	1	0	
1	1	0	0	1	
2	0	0	0	1	
3	0	0	1	0	
4	0	1	0	0	
...	...	...	...	...	
22618	0	0	0	1	
22619	0	0	0	1	
22620	0	0	0	1	
22621	0	0	0	1	
22622	0	0	0	1	

	dst_industrial	hod_sin	hod_cos
0	0	0.000000	1.000000
1	0	0.707107	-0.707107
2	0	-0.866025	0.500000
3	0	-1.000000	-0.000000
4	0	-1.000000	-0.000000
...	...	...	...
22618	0	0.707107	0.707107
22619	0	0.866025	-0.500000
22620	0	-0.258819	-0.965926
22621	0	-1.000000	-0.000000
22622	0	-0.258819	0.965926

[22579 rows x 24 columns]

[]:

ML_Prediction

April 13, 2026

1 Import Required Libraries for Data Processing and Modeling

This cell imports all necessary libraries for the workflow, including:

- **pandas** and **numpy** for data manipulation and numerical operations,
- **xgboost** for gradient boosting-based predictive modeling,
- **os** for file handling,
- **scikit-learn utilities** for dataset splitting and evaluation metrics
- **json** for handling structured data files.

These libraries support data preprocessing, feature engineering, model training, and performance evaluation.

```
[1]: import pandas as pd, numpy as np, xgboost as xgb, os
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
import json
```

2 Define Model Features, Target Variable, and XGBoost Parameters

- **FEATURES**: List of input variables capturing spatial (source/destination), temporal (hour, peak indicators, cyclic encoding), seasonal, distance, speed, and zone-type characteristics.
- **TARGET**: The variable to be predicted, i.e., mean travel time in minutes.
- **XGB_PARAMS**: Hyperparameters for the XGBoost model, controlling tree complexity, learning rate, sampling, regularization, and reproducibility.

```
[2]: FEATURES = ["source_id", "dest_id", "hod", "hod_sin", "hod_cos",
                "is_peak_morning", "is_peak_evening", "is_off_peak",
                "is_monsoon", "quarter", "distance_km", "speed_proxy",
                "src_it", "src_commercial", "src_residential", "src_industrial",
                "dst_it", "dst_commercial", "dst_residential", "dst_industrial",
                ]

TARGET = "mean_travel_time_min"

XGB_PARAMS = {"n_estimators":500, "max_depth":6, "learning_rate":0.05,
              "subsample":0.8, "colsample_bytree":0.8, "min_child_weight":5,
```

```

        "reg_alpha":0.1, "reg_lambda":1.0, "tree_method":"hist",
        ↪"random_state":42
    }

```

3 Define Evaluation Metrics for Quantile Regression

- **Pinball Loss:** Measures the accuracy of quantile predictions, penalizing over- and under-estimation asymmetrically based on the quantile level ().
- **PICP (Prediction Interval Coverage Probability):** Computes the proportion of true values that fall within the predicted interval, indicating coverage reliability.
- **PINAW (Prediction Interval Normalized Average Width):** Measures the average width of prediction intervals normalized by the data range, reflecting interval sharpness.

```

[3]: # Pinball (quantile) loss function.
def pinball_loss(y_true, y_pred, tau):
    errors = y_true - y_pred
    return np.mean(np.where(errors >= 0, tau * errors, (tau - 1) * errors))

# Prediction Interval Coverage Probability.
def picp(y_true, y_lower, y_upper):
    return np.mean((y_true >= y_lower) & (y_true <= y_upper))

# Prediction Interval Normalized Average Width.
def pinaw(y_lower, y_upper, y_range):
    return np.mean(y_upper - y_lower) / y_range

```

4 Train Quantile XGBoost Models and Evaluate Predictive Performance

This cell defines and runs the `train()` function to train quantile-based XGBoost models for travel time prediction.

The workflow includes: - loading the final processed dataset, - separating input features and target travel time, - splitting the data into training, validation, and test sets using a 70:15:15 ratio, - training three XGBoost quantile regression models for the 10th, 50th, and 90th percentiles, - applying early stopping using the validation set, - generating predictions on the test set, - evaluating the median model using **MAE**, **RMSE**, and **MAPE**, - assessing interval quality using **PICP** and **PINAW**, - computing **pinball loss** for all three quantiles, - extracting feature importance from the median model.

```

[4]: def train():
    df = pd.read_csv("data_processed.csv")
    X = df[FEATURES]
    y = df[TARGET]

    # 70 / 15 / 15 split

```

```

X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.15,
↪random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
↪test_size=0.1765, random_state=42)
print(f"  Train:{len(X_train):,} | Val:{len(X_val):,} | Test:{len(X_test):,}
↪,}")

models = {}
preds_test = {}

for q in [0.10, 0.50, 0.90]:
    print(f"  Training Q{int(q*100)} model...")
    model = xgb.XGBRegressor(objective="reg:quantileerror",
↪quantile_alpha=q, early_stopping_rounds=50, **XGB_PARAMS)
    model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)
    path = os.path.join("models", f"model_q{int(q*100)}.json")
    model.save_model(path)
    print(f"    Saved: {path} | Best iter: {model.best_iteration}")
    models[q] = model; preds_test[q] = model.predict(X_test)

# Convert y_test once
y_arr = y_test.values

# Safe MAPE function to avoid division-by-zero issues
def safe_mape(y_true, y_pred):
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    mask = y_true != 0
    if mask.sum() == 0:
        return np.nan
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) *
↪100

# Metrics for each quantile model
mae_q10 = mean_absolute_error(y_arr, preds_test[0.10])
rmse_q10 = np.sqrt(mean_squared_error(y_arr, preds_test[0.10]))
mape_q10 = safe_mape(y_arr, preds_test[0.10])

mae_q50 = mean_absolute_error(y_arr, preds_test[0.50])
rmse_q50 = np.sqrt(mean_squared_error(y_arr, preds_test[0.50]))
mape_q50 = safe_mape(y_arr, preds_test[0.50])

mae_q90 = mean_absolute_error(y_arr, preds_test[0.90])
rmse_q90 = np.sqrt(mean_squared_error(y_arr, preds_test[0.90]))
mape_q90 = safe_mape(y_arr, preds_test[0.90])

# Interval / quantile-specific metrics

```



```

cov = picp(y_arr, preds_test[0.10], preds_test[0.90])
wid = pinaw(preds_test[0.10], preds_test[0.90], y_arr.max()-y_arr.min())
pb10 = pinball_loss(y_arr, preds_test[0.10], 0.10)
pb50 = pinball_loss(y_arr, preds_test[0.50], 0.50)
pb90 = pinball_loss(y_arr, preds_test[0.90], 0.90)

print("\nTest Metrics:")
print(f" Q10 -> MAE:{mae_q10:.3f} min | RMSE:{rmse_q10:.3f} min | MAPE:
↪{mape_q10:.2f}%")
print(f" Q50 -> MAE:{mae_q50:.3f} min | RMSE:{rmse_q50:.3f} min | MAPE:
↪{mape_q50:.2f}%")
print(f" Q90 -> MAE:{mae_q90:.3f} min | RMSE:{rmse_q90:.3f} min | MAPE:
↪{mape_q90:.2f}%")
print(f" PICP:{cov*100:.1f}% | PINAW:{wid:.4f}")
print(f" Pinball Q10:{pb10:.4f} | Q50:{pb50:.4f} | Q90:{pb90:.4f}")

# Feature importance
fi_q10 = pd.Series(models[0.10].feature_importances_, index=FEATURES).
↪sort_values(ascending=False)
print(" Top 5 features for Q10:", list(fi_q10.head(5).index))
fi_q50 = pd.Series(models[0.50].feature_importances_, index=FEATURES).
↪sort_values(ascending=False)
print(" Top 5 features for Q50:", list(fi_q50.head(5).index))
fi_q90 = pd.Series(models[0.90].feature_importances_, index=FEATURES).
↪sort_values(ascending=False)
print(" Top 5 features for Q90:", list(fi_q90.head(5).index))

# Save metrics
metrics = {"MAE_Q10_min": round(mae_q10, 3),
           "RMSE_Q10_min": round(rmse_q10, 3),
           "MAPE_Q10_pct": round(mape_q10, 2),

           "MAE_Q50_min": round(mae_q50, 3),
           "RMSE_Q50_min": round(rmse_q50, 3),
           "MAPE_Q50_pct": round(mape_q50, 2),

           "MAE_Q90_min": round(mae_q90, 3),
           "RMSE_Q90_min": round(rmse_q90, 3),
           "MAPE_Q90_pct": round(mape_q90, 2),

           "PICP_pct":round(cov*100,1),"PINAW":round(wid,4),
           "PB_Q10":round(pb10,4), "PB_Q50":round(pb50,4), "PB_Q90":
↪round(pb90,4)}

pd.DataFrame([metrics]).to_csv("ml_metrics.csv",index=False)

```

```
train()
```

```
Train:15,804 | Val:3,388 | Test:3,387
Training Q10 model...
  Saved: models\model_q10.json | Best iter: 499
Training Q50 model...
  Saved: models\model_q50.json | Best iter: 499
Training Q90 model...
  Saved: models\model_q90.json | Best iter: 499
```

Test Metrics:

```
Q10 -> MAE:0.950 min | RMSE:2.482 min | MAPE:2.10%
Q50 -> MAE:0.388 min | RMSE:0.583 min | MAPE:1.19%
Q90 -> MAE:0.881 min | RMSE:1.279 min | MAPE:3.50%
PICP:73.0% | PINAW:0.0153
Pinball Q10:0.1149 | Q50:0.1942 | Q90:0.1194
Top 5 features for Q10: ['speed_proxy', 'distance_km', 'is_peak_evening',
'dst_commercial', 'is_off_peak']
Top 5 features for Q50: ['distance_km', 'speed_proxy', 'src_commercial',
'dst_commercial', 'is_peak_evening']
Top 5 features for Q90: ['distance_km', 'speed_proxy', 'is_peak_evening',
'is_off_peak', 'dst_commercial']
```

```
[5]: met = pd.read_csv("ml_metrics.csv")
met
```

```
[5]:  MAE_Q10_min  RMSE_Q10_min  MAPE_Q10_pct  MAE_Q50_min  RMSE_Q50_min  \
0          0.95         2.482          2.1         0.388         0.583

      MAPE_Q50_pct  MAE_Q90_min  RMSE_Q90_min  MAPE_Q90_pct  PICP_pct  PINAW  \
0          1.19         0.881         1.279          3.5        73.0  0.0153

      PB_Q10  PB_Q50  PB_Q90
0  0.1149  0.1942  0.1194
```

```
[6]: ZONES = ["Whitefield","Koramangala","Indiranagar","Hebbal","Marathahalli",
            "Electronic City","Jayanagar","Rajajinagar","Yeshwanthpur","BTM_
↳Layout",
            "HSR Layout","Bannerghatta Rd","Yelahanka","Sarjapur Road","MG_
↳Road","Banashankari"]

ZTYPE = {"Whitefield":"IT","Koramangala":"Commercial","Indiranagar":
↳"Commercial",
         "Hebbal":"Residential","Marathahalli":"IT","Electronic City":"IT",_
↳"Jayanagar":"Residential",
         "Rajajinagar":"Residential","Yeshwanthpur":"Industrial", "BTM Layout":
↳"Residential",
```

```

        "HSR Layout": "Commercial", "Bannerghatta Rd": "Residential", "Yelahanka":
↪ "Residential",
        "Sarjapur Road": "IT", "MG Road": "Commercial", "Banashankari":
↪ "Residential"}

```

5 Generate Travel Time Matrices Using Trained Quantile Models

This cell defines and executes the `generate_matrices()` function to construct origin-destination travel time matrices for all zone pairs under specified temporal conditions.

The workflow includes: - loading the distance matrix and trained quantile XGBoost models (Q10, Q50, Q90), - generating feature vectors for all valid OD pairs (excluding self-loops), - incorporating spatial, temporal, and zone-type features, - predicting travel times for each quantile while enforcing a minimum threshold, - constructing 16×16 matrices for Q10 (optimistic), Q50 (median), and Q90 (conservative) travel times, - extracting empirical mean travel times from the dataset for comparison.

```

[7]: def generate_matrices(hod=9, is_monsoon=0, quarter=4):
    n = len(ZONES)
    dist_df = pd.read_csv("distance_matrix_osrm.csv", index_col=0)

    # Load trained models
    models = {}
    for qi in [10, 50, 90]:
        m = xgb.XGBRegressor()
        m.load_model(os.path.join("models", f"model_q{qi}.json"))
        models[qi] = m

    # Build feature rows for all 240 OD pairs
    rows = []
    for i in range(n):
        for j in range(n):
            if i == j: continue
            d = dist_df.iloc[i, j]
            st = ZTYPE[ZONES[i]]; dt = ZTYPE[ZONES[j]]
            rows.append({
                "source_id": i, "dest_id": j, "hod": hod,
                "hod_sin": np.sin(2*np.pi*hod/24),
                "hod_cos": np.cos(2*np.pi*hod/24),
                "is_peak_morning": int(hod in [7,8,9,10]),
                "is_peak_evening": int(hod in [17,18,19,20]),
                "is_off_peak": int(hod not in [7,8,9,10,17,18,19,20]),
                "is_monsoon": is_monsoon, "quarter": quarter,
                "distance_km": d, "speed_proxy": d/0.55,
                "src_it": int(st=="IT"), "src_commercial": int(st=="Commercial"),
                "src_residential": int(st=="Residential"), "src_industrial":
↪ int(st=="Industrial"),

```

```

        "dst_it":int(dt=="IT"), "dst_commercial":int(dt=="Commercial"),
        "dst_residential":int(dt=="Residential"), "dst_industrial":
↪int(dt=="Industrial")
    })

X = pd.DataFrame(rows)[FEATURES]

# Predict all quantiles
preds = {}
for qi, m in models.items():
    preds[qi] = np.maximum(m.predict(X), 1.0)

# Build 16x16 matrices
q10 = np.zeros((n,n)); q50 = np.zeros((n,n)); q90 = np.zeros((n,n))
idx = 0
for i in range(n):
    for j in range(n):
        if i == j: continue
        q10[i][j] = round(preds[10][idx], 2)
        q50[i][j] = round(preds[50][idx], 2)
        q90[i][j] = round(preds[90][idx], 2)
        idx += 1

# Extract Uber mean travel times for reference
df_uber = pd.read_csv("data_processed.csv")
df_hod = df_uber[df_uber["hod"] == hod]
mean_m = np.zeros((n,n))
for i in range(n):
    for j in range(n):
        if i == j: continue
        m2 = df_hod[(df_hod["source_id"]==i) & (df_hod["dest_id"]==j)]
        mean_m[i][j] = round(m2["mean_travel_time_min"].mean() if len(m2)
↪else q50[i][j], 2)

out = {"hod":hod, "zones":ZONES, "Q10":q10.tolist(), "Q50":q50.tolist(),
        "Q90":q90.tolist(), "mean_uber":mean_m.tolist()}

with open("travel_time_matrices.json", "w") as f: json.dump(out, f,
↪indent=2)
print(f" Saved: {'travel_time_matrices.json'}")

generate_matrices(hod=9, is_monsoon=0, quarter=4)

```

Saved: travel_time_matrices.json

CVRP_Solver

April 14, 2026

1 Import Libraries for Routing Optimization

This cell imports the required libraries for solving the vehicle routing problem (VRP).

OR-Tools Routing Solver modules are imported to model and solve routing optimization problems using Google's OR-Tools framework.

```
[1]: import json
import numpy as np
import os
from ortools.constraint_solver import routing_enums_pb2, pywrapcp
```

2 Define CVRP Optimization Parameters

- **NUM_VEHICLES**: Total number of delivery vehicles available.
- **VEHICLE_CAPACITY**: Maximum load capacity assigned to each vehicle.
- **DEPOT_INDEX**: Index of the depot/starting location in the zone list.
- **SCALE**: Multiplier used to convert travel times into integers, as required by OR-Tools.
- **SOLVER_TIME_LIMIT**: Maximum allowed computation time (in seconds) for each optimization run.

```
[2]: # --- CVRP PARAMETERS ---
NUM_VEHICLES      = 4
VEHICLE_CAPACITY  = 100    # units per vehicle
DEPOT_INDEX       = 0      # Whitefield
SCALE             = 100    # OR-Tools needs integers - multiply minutes x 100
SOLVER_TIME_LIMIT = 30     # seconds per instance
```

3 Generate a Random CVRP Demand Instance

This cell defines the `create_cvrp_instance()` function to create a synthetic Capacitated Vehicle Routing Problem (CVRP) instance with randomized customer demands.

The function performs the following steps:

- sets a random seed for reproducibility,
- creates one depot node with zero demand,
- generates random customer demands within the specified range,
- returns the total number of nodes along with the demand vector.

```
[3]: # Generate a CVRP instance with random demands.
def create_cvrp_instance(n_customers=15, seed=42):
    np.random.seed(seed)
    demands = [0] + [int(np.random.randint(10, 31)) for _ in range(n_customers)]
    return {"n_nodes": n_customers + 1, "demands": demands}
```

4 Solve the CVRP Using Google OR-Tools

This cell defines the `solve_cvrp()` function to solve a Capacitated Vehicle Routing Problem (CVRP) instance using Google OR-Tools.

The function performs the following tasks:

- converts the travel-time cost matrix into scaled integer values required by OR-Tools,
- creates the routing index manager and routing model,
- defines a travel-cost callback based on the selected travel-time matrix,
- defines a demand callback for customer load values,
- sets travel time as the arc cost for all vehicles,
- adds vehicle capacity constraints,
- allows customer dropping with a large penalty to handle infeasible cases,
- configures the solver using **Path Cheapest Arc** for initial solution generation and **Guided Local Search** for route improvement,
- solves the routing problem within the specified time limit,
- extracts the optimized routes, vehicle loads, and route costs from the solution,
- returns a structured summary including feasibility status, total cost, number of vehicles used, and route details.

```
[4]: # Solve CVRP using Google OR-Tools.
def solve_cvrp(cost_matrix, instance, label=""):
    n = instance["n_nodes"]
    dem = instance["demands"]

    # Scale travel times to integers
    ci = [[int(cost_matrix[i][j] * SCALE) for j in range(n)] for i in range(n)]

    # Create routing model
    manager = pywrapcp.RoutingIndexManager(n, NUM_VEHICLES, DEPOT_INDEX)
    routing = pywrapcp.RoutingModel(manager)

    # Cost callback - uses Q10, Q50, or Q90 travel times
    def cost_callback(from_idx, to_idx):
        i = manager.IndexToNode(from_idx)
        j = manager.IndexToNode(to_idx)
        return ci[i][j]

    # Demand callback for capacity dimension
    def demand_callback(from_idx):
        return dem[manager.IndexToNode(from_idx)]
```

```

cost_cb_idx    = routing.RegisterTransitCallback(cost_callback)
demand_cb_idx = routing.RegisterUnaryTransitCallback(demand_callback)

# Set arc cost
routing.SetArcCostEvaluatorOfAllVehicles(cost_cb_idx)

# Capacity dimension
routing.AddDimensionWithVehicleCapacity(
    demand_cb_idx,
    0,                                     # no slack
    [VEHICLE_CAPACITY] * NUM_VEHICLES,   # vehicle capacities
    True,                                 # start cumul at zero
    "Capacity"
)

# Allow dropping customers with a high penalty (handles infeasible cases)
penalty = 100000
for node in range(1, n):
    routing.AddDisjunction([manager.NodeToIndex(node)], penalty)

# Solver settings - Guided Local Search
search_params = pywrapcp.DefaultRoutingSearchParameters()
search_params.first_solution_strategy = routing_enums_pb2.
↳FirstSolutionStrategy.PATH_CHEAPEST_ARC
search_params.local_search_metaheuristic = routing_enums_pb2.
↳LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
search_params.time_limit.seconds = SOLVER_TIME_LIMIT
search_params.log_search = False

solution = routing.SolveWithParameters(search_params)
if not solution:
    return {"status": "INFEASIBLE", "label": label, "routes": [], "total_cost":
↳9999, "n_vehicles_used": 0}

# Extract routes from solution
routes = []
total_cost = 0
for v in range(NUM_VEHICLES):
    route = []
    idx = routing.Start(v)
    rc = 0
    while not routing.IsEnd(idx):
        node = manager.IndexToNode(idx)
        route.append(node)
        next_idx = solution.Value(routing.NextVar(idx))
        rc += ci[manager.IndexToNode(idx)][manager.IndexToNode(next_idx)]

```

```

        idx = next_idx
        route.append(manager.IndexToNode(idx)) # return to depot
        if len(route) > 2: # non-empty route
            load = sum(dem[nd] for nd in route if nd != DEPOT_INDEX)
            routes.append({
                "demands": dem,
                "vehicle": v + 1,
                "route": route,
                "cost_min": round(rc / SCALE, 2),
                "load": load
            })
            total_cost += rc

    return {"status": "FEASIBLE", "label": label, "routes": routes, "total_cost":
    ↪round(total_cost/SCALE,2), "n_vehicles_used": len(routes)}

```

```

[5]: # Print routes with zone names.
def print_routes(result, zones):
    print(f" Strategy: {result['label']} | Total: {result['total_cost']}␣
    ↪min")
    for r in result["routes"]:
        names = [zones[nd] for nd in r["route"]]
        print(f" V{r['vehicle']}: {' -> '.join(names)}")
        print(f" Load: {r['load']}/{VEHICLE_CAPACITY} Cost:␣
    ↪{r['cost_min']} min")

```

5 Run the CVRP Solver Across Multiple Quantile-Based Travel Time Scenarios

This cell defines the `run_solver()` function to evaluate routing performance under three uncertainty-aware travel-time strategies: **Q10**, **Q50**, and **Q90**.

The workflow includes: - loading the previously generated travel-time matrices, - extracting the zone list and selected hour of day, - generating 20 random CVRP demand instances using different seeds, - solving each instance separately using the Q10, Q50, and Q90 travel-time matrices, - aggregating total travel cost and vehicle usage across all feasible runs, - computing the average cost for each strategy, - comparing the percentage cost gaps between conservative, median, and optimistic scenarios.

```

[6]: def run_solver():
    print("SOLVING CVRP - THREE QUANTILE STRATEGIES")

    # Load travel time matrices
    with open("travel_time_matrices.json") as f: matrices = json.load(f)
    zones = matrices["zones"]
    q10 = matrices["Q10"]
    q50 = matrices["Q50"]

```



```

q90      = matrices["Q90"]
hod      = matrices["hod"]
print(f"  Hour: {hod}:00 | Zones: {len(zones)} | Vehicles: {NUM_VEHICLES}")

all_q10 = []; all_q50 = []; all_q90 = []; demands = []

print(f"\n  Running 20 instances per strategy...")
for seed in range(20):
    inst = create_cvrp_instance(n_customers=15, seed=seed)

    r10 = solve_cvrp(q10, inst, f"Q10 seed={seed}")
    r50 = solve_cvrp(q50, inst, f"Q50 seed={seed}")
    r90 = solve_cvrp(q90, inst, f"Q90 seed={seed}")

    all_q10.append(r10); all_q50.append(r50); all_q90.append(r90); demands.
    ↪.append(inst["demands"])

    # Print first instance in detail
    if seed == 0:
        print("\n  --- Instance 1 (seed=0) detailed routes ---")
        print(demands[0])
        print_routes(r10, zones)
        print()
        print_routes(r50, zones)
        print()
        print_routes(r90, zones)

    if seed == 2:
        print("\n  --- Instance 2 (seed=2) detailed routes ---")
        print(demands[2])
        print_routes(r10, zones)
        print()
        print_routes(r50, zones)
        print()
        print_routes(r90, zones)

    # Aggregate results
    c10 = [r["total_cost"] for r in all_q10 if r["status"]=="FEASIBLE"]
    c50 = [r["total_cost"] for r in all_q50 if r["status"]=="FEASIBLE"]
    c90 = [r["total_cost"] for r in all_q90 if r["status"]=="FEASIBLE"]
    v10 = [r["n_vehicles_used"] for r in all_q10 if r["status"]=="FEASIBLE"]
    v50 = [r["n_vehicles_used"] for r in all_q50 if r["status"]=="FEASIBLE"]
    v90 = [r["n_vehicles_used"] for r in all_q90 if r["status"]=="FEASIBLE"]

    print(f"\n  Summary across 20 instances:")
    print(f"    Q10 avg cost: {np.mean(c10):.1f} min | min vehicles: {np.min(v10):
    ↪.1f} | max vehicles: {np.max(v10):.1f}")

```

```

    print(f" Q50 avg cost: {np.mean(c50):.1f} min | min vehicles: {np.min(v50):
↵.1f} | max vehicles: {np.max(v50):.1f}")
    print(f" Q90 avg cost: {np.mean(c90):.1f} min | min vehicles: {np.min(v90):
↵.1f} | max vehicles: {np.max(v90):.1f}")
    gap50 = (np.mean(c50)-np.mean(c10))/np.mean(c10)*100
    gap90 = (np.mean(c90)-np.mean(c10))/np.mean(c10)*100
    print(f" Q50 vs Q10 cost gap: {gap50:+.1f}%")
    print(f" Q90 vs Q10 cost gap: {gap90:+.1f}%")

    output = {
        "hod": hod, "n_instances": 20,
        "q10_results": all_q10, "q50_results": all_q50, "q90_results": all_q90,
        "summary": {
            "q10_avg_cost": round(np.mean(c10), 2),
            "q50_avg_cost": round(np.mean(c50), 2),
            "q90_avg_cost": round(np.mean(c90), 2),
            "q10_avg_vehicles": round(np.mean(v10), 2),
            "q50_avg_vehicles": round(np.mean(v50), 2),
            "q90_avg_vehicles": round(np.mean(v90), 2),
            "q50_vs_q10_pct": round(gap50, 2),
            "q90_vs_q10_pct": round(gap90, 2)
        }
    }
    with open("cvrp_routes.json", "w") as f: json.dump(output, f, indent=2)
    print(f" Saved: {"cvrp_routes.json"}")

```

```
[7]: run_solver()
```

SOLVING CVRP - THREE QUANTILE STRATEGIES

Hour: 9:00 | Zones: 16 | Vehicles: 4

Running 20 instances per strategy...

--- Instance 1 (seed=0) detailed routes ---

[0, 22, 25, 10, 13, 13, 17, 19, 29, 28, 14, 16, 22, 11, 16, 17]

Strategy: Q10 seed=0 | Total: 558.26 min

V2: Whitefield -> Hebbal -> Yelahanka -> Yeshwanthpur -> Rajajinagar -> Marathahalli -> Whitefield

Load: 93/100 Cost: 198.03 min

V3: Whitefield -> Sarjapur Road -> Electronic City -> Bannerghatta Rd -> MG Road -> Indiranagar -> Whitefield

Load: 81/100 Cost: 184.81 min

V4: Whitefield -> Banashankari -> Jayanagar -> BTM Layout -> HSR Layout -> Koramangala -> Whitefield

Load: 98/100 Cost: 175.42 min

Strategy: Q50 seed=0 | Total: 568.79 min

V2: Whitefield -> Sarjapur Road -> Banashankari -> Jayanagar -> BTM Layout -> HSR Layout -> Whitefield

Load: 87/100 Cost: 179.65 min

V3: Whitefield -> Hebbal -> Yelahanka -> Yeshwanthpur -> Rajajinagar -> Marathahalli -> Whitefield

Load: 93/100 Cost: 199.68 min

V4: Whitefield -> Koramangala -> Electronic City -> Bannerghatta Rd -> MG Road -> Indiranagar -> Whitefield

Load: 92/100 Cost: 189.46 min

Strategy: Q90 seed=0 | Total: 582.4 min

V1: Whitefield -> Yeshwanthpur -> Rajajinagar -> Yelahanka -> Hebbal -> Whitefield

Load: 80/100 Cost: 174.62 min

V2: Whitefield -> Sarjapur Road -> Banashankari -> Jayanagar -> MG Road -> Indiranagar -> Marathahalli -> Whitefield

Load: 99/100 Cost: 219.91 min

V3: Whitefield -> BTM Layout -> HSR Layout -> Koramangala -> Bannerghatta Rd -> Electronic City -> Whitefield

Load: 93/100 Cost: 187.87 min

--- Instance 2 (seed=2) detailed routes ---

[0, 18, 25, 23, 18, 21, 28, 21, 18, 17, 12, 27, 21, 25, 30, 30]

Strategy: Q10 seed=2 | Total: 599.72 min

V1: Whitefield -> Hebbal -> Yelahanka -> Yeshwanthpur -> Rajajinagar -> Whitefield

Load: 83/100 Cost: 168.31 min

V2: Whitefield -> Koramangala -> Electronic City -> Bannerghatta Rd -> Sarjapur Road -> Whitefield

Load: 91/100 Cost: 162.51 min

V3: Whitefield -> Marathahalli -> MG Road -> Indiranagar -> Whitefield

Load: 73/100 Cost: 120.06 min

V4: Whitefield -> Banashankari -> Jayanagar -> BTM Layout -> HSR Layout -> Whitefield

Load: 87/100 Cost: 148.84 min

Strategy: Q50 seed=2 | Total: 609.13 min

V1: Whitefield -> Sarjapur Road -> Banashankari -> Bannerghatta Rd -> Marathahalli -> Whitefield

Load: 100/100 Cost: 169.27 min

V2: Whitefield -> Hebbal -> Yelahanka -> Yeshwanthpur -> Rajajinagar -> Whitefield

Load: 83/100 Cost: 169.6 min

V3: Whitefield -> Electronic City -> Jayanagar -> BTM Layout -> HSR Layout -> Koramangala -> Whitefield

Load: 96/100 Cost: 178.18 min

V4: Whitefield -> MG Road -> Indiranagar -> Whitefield

Load: 55/100 Cost: 92.08 min

Strategy: Q90 seed=2 | Total: 621.07 min
V1: Whitefield -> Yeshwanthpur -> Rajajinagar -> Yelahanka -> Hebbal ->
Whitefield
Load: 83/100 Cost: 174.62 min
V2: Whitefield -> Sarjapur Road -> Banashankari -> Jayanagar -> Whitefield
Load: 83/100 Cost: 135.43 min
V3: Whitefield -> BTM Layout -> HSR Layout -> Koramangala -> Bannerghatta Rd
-> Electronic City -> Whitefield
Load: 95/100 Cost: 187.87 min
V4: Whitefield -> MG Road -> Indiranagar -> Marathahalli -> Whitefield
Load: 73/100 Cost: 123.15 min

Summary across 20 instances:

Q10 avg cost: 575.9 min | min vehicles: 3.0 | max vehicles: 4.0

Q50 avg cost: 586.1 min | min vehicles: 3.0 | max vehicles: 4.0

Q90 avg cost: 599.0 min | min vehicles: 3.0 | max vehicles: 4.0

Q50 vs Q10 cost gap: +1.8%

Q90 vs Q10 cost gap: +4.0%

Saved: cvrp_routes.json

analyse

April 14, 2026

```
[1]: import json
import numpy as np
import os
from ortools.constraint_solver import routing_enums_pb2, pywrapcp
import pandas as pd
import xgboost as xgb
```

```
[2]: NUM_VEHICLES = 4
CAP = 100
DEPOT = 0
SCALE = 100
TL = 30
```

```
[3]: ZONES = ["Whitefield", "Koramangala", "Indiranagar", "Hebbal", "Marathahalli",
             "Electronic City", "Jayanagar", "Rajajinagar", "Yeshwanthpur", "BTM_Layout",
             ↪ "Layout",
             "HSR Layout", "Bannerghatta Rd", "Yelahanka", "Sarjapur Road", "MG_Road",
             ↪ "Road", "Banashankari"]

ZTYPE = {"Whitefield": "IT", "Koramangala": "Commercial", "Indiranagar":
        ↪ "Commercial",
        "Hebbal": "Residential", "Marathahalli": "IT", "Electronic City": "IT",
        "Jayanagar": "Residential", "Rajajinagar": "Residential", "Yeshwanthpur":
        ↪ "Industrial",
        "BTM Layout": "Residential", "HSR Layout": "Commercial", "Bannerghatta Rd":
        ↪ "Residential",
        "Yelahanka": "Residential", "Sarjapur Road": "IT", "MG Road": "Commercial",
        "Banashankari": "Residential"}

FEATURES = ["source_id", "dest_id", "hod", "hod_sin", "hod_cos", "is_peak_morning",
            "is_peak_evening", "is_off_peak", "is_monsoon", "quarter",
            ↪
            ↪ "distance_km", "speed_proxy", "src_it", "src_commercial", "src_residential",
            ↪
            ↪ "src_industrial", "dst_it", "dst_commercial", "dst_residential", "dst_industrial"]
```

1 Generate Hour-Specific Quantile Travel Time Matrices

This cell defines the `get_matrices_for_hour()` function to generate **Q10**, **Q50**, and **Q90** travel-time matrices which are used to analyze how routing outcomes change across hours and uncertainty levels.

```
[4]: # Generate Q10/Q50/Q90 matrices for a given hour using trained models.
def get_matrices_for_hour(hod, dist_df):
    models = {}
    for qi in [10, 50, 90]:
        m = xgb.XGBRegressor()
        m.load_model(os.path.join("models", f"model_q{qi}.json"))
        models[qi] = m

    n = len(ZONES)
    rows = []
    for i in range(n):
        for j in range(n):
            if i == j: continue
            d = dist_df.iloc[i,j]
            st = ZTYPE[ZONES[i]]; dt = ZTYPE[ZONES[j]]
            rows.append({
                "source_id":i,"dest_id":j,"hod":hod,
                "hod_sin":np.sin(2*np.pi*hod/24),"hod_cos":np.cos(2*np.pi*hod/
↪24),

                "is_peak_morning":int(hod in[7,8,9,10]),
                "is_peak_evening":int(hod in[17,18,19,20]),
                "is_off_peak":int(hod not in[7,8,9,10,17,18,19,20]),
                "is_monsoon":1,"quarter":2,
                "distance_km":d,"speed_proxy":d/0.55,
                "src_it":int(st=="IT"),"src_commercial":int(st=="Commercial"),
                "src_residential":int(st=="Residential"),"src_industrial":
↪int(st=="Industrial"),
                "dst_it":int(dt=="IT"),"dst_commercial":int(dt=="Commercial"),
                "dst_residential":int(dt=="Residential"),"dst_industrial":
↪int(dt=="Industrial")
            })

    X = pd.DataFrame(rows)[FEATURES]
    preds = {}
    for qi, m in models.items():
        preds[qi] = np.maximum(m.predict(X), 1.0)

    q10=np.zeros((n,n)); q50=np.zeros((n,n)); q90=np.zeros((n,n))
    idx=0
    for i in range(n):
        for j in range(n):
```

```

        if i==j: continue
        q10[i][j]=round(preds[10][idx],2)
        q50[i][j]=round(preds[50][idx],2)
        q90[i][j]=round(preds[90][idx],2)
        idx+=1
    return q10.tolist(), q50.tolist(), q90.tolist()

```

2 Solve the CVRP for Comparative Analysis

This cell defines the `solve_cvrp()` function used for repeated routing analysis across different travel-time matrices.

```

[5]: # CVRP solve for analysis.
def solve_cvrp(cm, n_customers=15, seed=0):
    np.random.seed(seed)
    n = n_customers + 1
    dem = [0] + [int(np.random.randint(10,31)) for _ in range(n_customers)]
    ci = [[int(cm[i][j]*SCALE) for j in range(n)] for i in range(n)]
    mgr = pywrapcp.RoutingIndexManager(n, NUM_VEHICLES, DEPOT)
    rt = pywrapcp.RoutingModel(mgr)
    def cc(f,t): return ci[mgr.IndexToNode(f)][mgr.IndexToNode(t)]
    def dc(f): return dem[mgr.IndexToNode(f)]
    ci_ = rt.RegisterTransitCallback(cc)
    di_ = rt.RegisterUnaryTransitCallback(dc)
    rt.SetArcCostEvaluatorOfAllVehicles(ci_)
    rt.AddDimensionWithVehicleCapacity(di_, 0, [CAP]*NUM_VEHICLES, True, "Cap")
    for nd in range(1,n): rt.AddDisjunction([mgr.NodeToIndex(nd)], 100000)
    sp = pywrapcp.DefaultRoutingSearchParameters()
    sp.first_solution_strategy = routing_enums_pb2.FirstSolutionStrategy.
    ↪PATH_CHEAPEST_ARC
    sp.local_search_metaheuristic = routing_enums_pb2.LocalSearchMetaheuristic.
    ↪GUIDED_LOCAL_SEARCH
    sp.time_limit.seconds = TL; sp.log_search = False
    sol = rt.SolveWithParameters(sp)
    if not sol: return 9999
    total = 0

    for v in range(NUM_VEHICLES):
        idx = rt.Start(v)
        while not rt.IsEnd(idx):
            ni = sol.Value(rt.NextVar(idx))
            total += ci[mgr.IndexToNode(idx)][mgr.IndexToNode(ni)]; idx=ni
    return round(total/SCALE, 2)

```

3 Analyze the Cost–Uncertainty Tradeoff Across Different Times of Day

This cell defines and runs the `analyse()` function to study how routing cost changes under different travel-time uncertainty levels at selected hours of the day.

The workflow includes: - loading the inter-zone distance matrix, - selecting representative hourly time slots such as late night, morning peak, midday, and evening peak, - generating Q10, Q50, and Q90 travel-time matrices for each hour, - solving five random CVRP instances for each quantile scenario, - computing the average routing cost for optimistic, median, and conservative travel-time estimates, - measuring the percentage cost gap between Q90 and Q10, - printing the cost comparison for each time slot.

```
[6]: def analyse():
    print("COST-UNCERTAINTY TRADEOFF ANALYSIS")
    dist_df = pd.read_csv("distance_matrix_osrm.csv", index_col=0)
    # Test key hours: 2am, 7am, 9am, 12pm, 6pm, 10pm
    test_hours = [2, 7, 9, 12, 18, 22]
    labels      = ["Late night", "Pre-peak morning", "Peak morning", "Midday", "Peak_
evening", "Night"]
    results     = []
    for hod, label in zip(test_hours, labels):
        q10, q50, q90 = get_matrices_for_hour(hod, dist_df)
        # Solve 5 instances per hour and average
        c10 = np.mean([solve_cvrp(q10,15,s) for s in range(5)])
        c50 = np.mean([solve_cvrp(q50,15,s) for s in range(5)])
        c90 = np.mean([solve_cvrp(q90,15,s) for s in range(5)])
        gap = (c90-c10)/c10*100
        print(f" {label} (hod={hod}): Q10={c10:.1f} Q50={c50:.1f} Q90={c90:.
1f} gap={gap:.1f}%")
        results.append({
            "time_slot":label, "hod":hod,
            "q10_cost":round(c10,1), "q50_cost":round(c50,1),
            "q90_cost":round(c90,1), "q90_vs_q10_pct":round(gap,1)
        })
    df_results = pd.DataFrame(results)
    df_results.to_csv("cost_tradeoff_analysis.csv", index=False)
    print(f" Saved: {"cost_tradeoff_analysis.csv"}")
    return df_results
```

```
[7]: analyse()
```

COST-UNCERTAINTY TRADEOFF ANALYSIS

Late night (hod=2): Q10=520.8 Q50=525.4 Q90=557.1 gap=+7.0%
Pre-peak morning (hod=7): Q10=553.4 Q50=568.8 Q90=598.1 gap=+8.1%
Peak morning (hod=9): Q10=567.1 Q50=577.5 Q90=589.5 gap=+3.9%
Midday (hod=12): Q10=563.0 Q50=570.5 Q90=583.9 gap=+3.7%
Peak evening (hod=18): Q10=575.2 Q50=587.7 Q90=610.7 gap=+6.2%

Night (hod=22): Q10=542.1 Q50=550.2 Q90=574.0 gap=+5.9%
Saved: cost_tradeoff_analysis.csv

```
[7]:
```

	time_slot	hod	q10_cost	q50_cost	q90_cost	q90_vs_q10_pct
0	Late night	2	520.8	525.4	557.1	7.0
1	Pre-peak morning	7	553.4	568.8	598.1	8.1
2	Peak morning	9	567.1	577.5	589.5	3.9
3	Midday	12	563.0	570.5	583.9	3.7
4	Peak evening	18	575.2	587.7	610.7	6.2
5	Night	22	542.1	550.2	574.0	5.9